

Curve approximation by G^1 arc splines with a limited number of types of curvature and length [☆]



Keisuke Mizutani ^{*}, Ken'ichi Kawaguchi

ARTICLE INFO

Article history:

Received 1 April 2021

Received in revised form 19 July 2021

Accepted 25 August 2021

Available online 2 September 2021

Keywords:

Arc spline

Approximation

Optimization

Clustering

ABSTRACT

A method for approximating planar curves by G^1 continuous arc splines with a limited number of types of curvatures and lengths, named k-arc splines, is proposed. A k-arc spline is a particular arc spline, and it has only k types of arcs less than the total number of segments. Due to this property, k-arc splines require discrete variables that relate segments and k types of arcs for representation. Therefore k-arc spline approximation problems must deal with discrete variables in contrast to general arc spline approximation problems. The authors present a method to appropriately estimate these variables from curvature values of a target curve. The proposed method enables a formulation of the approximation problem as a continuous optimization problem that conventional optimization algorithms can solve efficiently. The estimation method also provides a suitable initial solution for the algorithms. Some examples to show the broad applicability of the proposed algorithm are shown.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Arc splines are G^1 continuous curves consisting of only arc segments and straight-line segments. They have been studied mainly for the application to NC toolpaths of CNC machines because they have many advantages such as stability of CNC machines during machining processes and improvement of processing speed compared to toolpaths consisting of only straight-line segments (Meek and Walton, 1993; Yeung and Walton, 1994). Approximation methods for discrete points (Chen et al., 2004; Hoschek, 1992; Meek and Walton, 1992; Parkinson and Moreton, 1991; Schonherr, 1993; Yeung and Walton, 1994; Yong et al., 1999) and planar curves (Ahn et al., 1998; Maier, 2014; Meek and Walton, 1993, 1995; Ong et al., 1996; Piegl and Tiller, 2002; Walton and Meek, 1994; Yeung and Walton, 1994) have been investigated. Most of them use the biarc curve introduced in Bolton, 1975, which is a pair of two arcs connected in G^1 manner, and it can interpolate arbitrary two points and tangent vectors at those two points, whereas one arc curve can only interpolate arbitrary two points and a tangent vector at one of them. They employ an approach that generates an arc spline by interpolating between target points with biarcs. They are different in extracting an interpolation point sequence, estimating tangent vectors at interpolation points, and the biarc 6th condition. Other approaches include a method directly representing arc splines in a rational Bézier form (Hoschek, 1992) and one limiting target curves to quadratic Bézier curves and assigning one arc to a sub-curve with a monotonous curvature (Ahn et al., 1998). In arc spline approximation problems, approximation accuracy and the number of segments generally indicate approximation quality, i.e., an arc spline that accurately approximates its target with a smaller number of segments is ideal considering engineering applications. For this purpose, two main approaches have been inves-

[☆] Editor: Javier Sánchez-Reyes.

^{*} Corresponding author. Postal address: #Bw-502, Institute of Industrial Science, the University of Tokyo, Komaba 4-6-1, Meguro-ku, Tokyo, Japan.

E-mail address: kei-mizu@iis.u-tokyo.ac.jp (K. Mizutani).

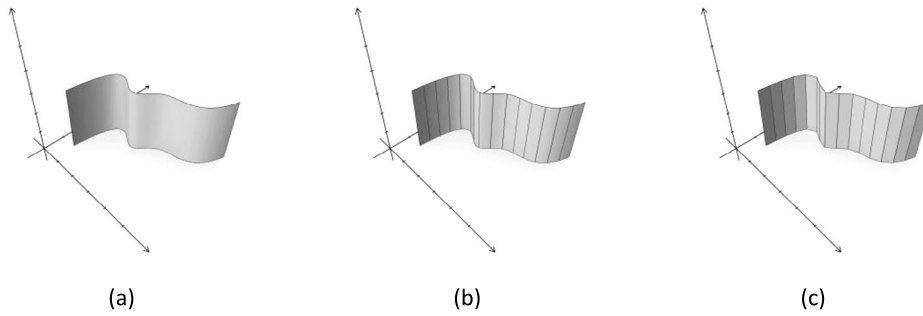


Fig. 1. Cylindrical surfaces segmentation. (a) A continuous cylindrical surface. (b) A smooth segmented surface by simple segmentation of its generatrix. (c) A non-smooth segmented surface by polygonization of its generatrix. In these segmentation patterns, surface segmentation problems reduce to segmentation problems of its generatrix.

tigated. One is to generate an arc spline with a smaller number of segments within a given tolerance, and another is to estimate an appropriate number of segments and then minimize the deviation between the arc spline and the target. In this paper, we adopt the second approach, i.e., the optimization approach.

Unlike the previous studies, however, we apply arc splines in architectural engineering. We propose a new arc spline approximation method considering conditions specific to architectural engineering. In recent years, architectural design using free-form surfaces has become a trend due to the spread of CAD software that enables intuitive control of curves and surfaces and the development of design theory. In this paper, we consider the application of arc splines to walls or roofs designed with free-form cylindrical surfaces. In general, large-scale surfaces designed by free-form continuous surfaces, e.g., Nurbs surfaces, in the conceptual design phase must be segmented into multiple small parts, called panels, in the design development phase because of some limitation of fabrication, construction, or transportation. This process is called “Paneling” in the architectural geometry field. Although the segmentation patterns vary depending on each project, we consider that cylindrical surfaces are segmented through generatrix segmentation shown in Fig. 1, which is the most general segmentation pattern of cylindrical surfaces. Each panel is manufactured in a way suitable for the material and shape of panels. In manufacturing, curved panels made of glass or pre-cast concrete often used in architectural design generally require molds. The manufacturing cost of one mold is often drastically higher than that of one panel, and it has a great effect on the construction cost of surfaces (Eigensatz et al., 2010). Since all panels obtained from extrusions of sub-curves of a free-form generatrix are different in shape, a unique mold is necessary for each panel, which leads to a massive increase in cost. Therefore, how to reduce the number of molds, i.e., how to increase the number of repetitive panels, is a critical issue for cost-effective design. For this problem, the conventional method approximates a cylindrical surface by polygonization of its generatrix (see Fig. 1) and designs all panels as flat panels which do not require specific molds. While this solution is the most cost-effective, it is not desirable in appearance because it causes creases at the panel connections and decreases the smoothness of designed surfaces. For the realization of ideal surfaces, a method that can solve the trade-off between cost and appearance is desirable. In this situation, approximating the generatrix with an arc spline is a reasonable solution because it is G^1 continuous, and all segments are arcs that can be subdivided into the same sub-arcs, which leads to repetition of the same curve elements. The general arc spline, however, consists of arc segments with different radius. Thus, it requires as many molds as the number of segments. In addition, the higher the approximation accuracy, the more segments it includes. As a result, short segments arise, the number of subdivisions reduces, and the repetition of the same elements decreases. They cannot promote repetitive elements drastically. Therefore, in this paper, we introduce a new arc spline defined as a k-arc spline. The k-arc splines satisfy special conditions for the repetition of the same elements. Finally, we present a k-arc spline approximation algorithm that enables users to control the number of element types explicitly.

The organization of the paper is as follows. In section 2 we give a formulation of arc splines and k-arc splines. In section 3 we present an approximation algorithm that combines a continuous optimization problem and an effective initialization algorithm. In section 4 results of some examples are given, and we finally conclude in section 5.

2. Preliminaries

2.1. Arc spline

In this paper, an arc spline is controlled with radii and lengths of segments as the main variables because of our specific purpose to constraint segment shapes. Considering that the deviation between an arc spline and a target curve is evaluated approximately by the sum of squares of the residual vectors at nodes and midpoints of segments in the optimization problem described in the next section, we describe representations of coordinates of nodes and midpoints of arc splines in this section.

Let us assume that an arc spline is on the x-y plane because we consider only planar curves. Let $\mathbf{P}_i$ ($i = 0, 1, \dots, n$) be a node of the arc spline, $\mathbf{P}_i^m$ ($i = 0, 1, \dots, n - 1$) be a middle point of the i-th arc segment, r_i ($i = 0, 1, \dots, n - 1$) be

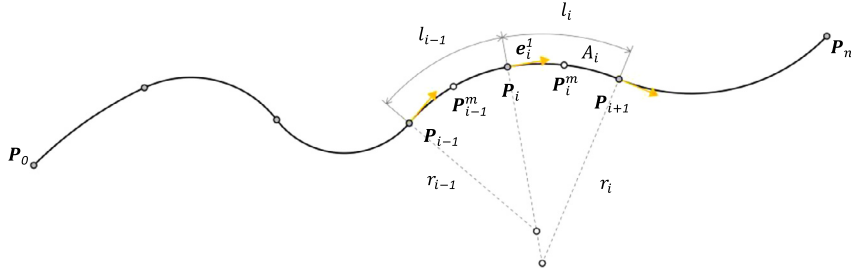


Fig. 2. The representation of arc splines. An arc spline is controlled by radii, lengths and binormal vectors of arc segments and a boundary condition.

a radius of the i -th segment, $\mathbf{e}_i^1$ ($i = 0, 1, \dots, n$) be a unit tangent vector at the i -th node and $\mathbf{e}_i^3 \in \{(0, 0, 1), (0, 0, -1)\}$ ($i = 0, 1, \dots, n-1$) be a unit binormal vector on the i -th arc segment as shown in Fig. 2, the relation between $\mathbf{P}_{i+1}$ and $\mathbf{P}_i$ is as follows,

$$\mathbf{P}_{i+1} = \mathbf{P}_i + r_i(\mathbf{e}_{i+1}^1 - \mathbf{e}_i^1) \times \mathbf{e}_i^3 \quad (i = 0, 1, \dots, n-1) \quad (1)$$

The relation between $\mathbf{P}_i$ and $\mathbf{P}_i^m$ is as follows,

$$\mathbf{P}_i^m = \mathbf{P}_i + r_i(\mathbf{e}_i^3 \times \mathbf{e}_i^1) - r_i \frac{\mathbf{e}_{i+1}^1 - \mathbf{e}_i^1}{\sqrt{2(1 - \langle \mathbf{e}_{i+1}^1, \mathbf{e}_i^1 \rangle)}} \quad (i = 0, 1, \dots, n-1) \quad (2)$$

The relation between $\mathbf{e}_{i+1}^1$ and $\mathbf{e}_i^1$ is as follows,

$$\mathbf{e}_{i+1}^1 = R\left(\mathbf{e}_{i,z}^3 \frac{l_i}{r_i}\right) \mathbf{e}_i^1 \quad (i = 0, 1, \dots, n-1) \quad (3)$$

Where a matrix $R(\cdot)$ is a rotation matrix on the x-y plane, $\mathbf{e}_{i,z}^3 \in \{-1, 1\}$ is the z component of the i -th unit binormal vector and l_i ($i = 0, 1, \dots, n-1$) is the length of the i -th segment. If we are given an initial condition, i.e., a starting point $\mathbf{P}_0$ and a unit tangent vector $\mathbf{e}_0^1$ at $\mathbf{P}_0$, and radii and lengths of all segments, we get all nodes and middle points of the arc spline.

2.2. k-arc spline

In this paper, we use special arc splines named k-arc splines. k-arc splines are defined as follows,

Definition 1. Let $A = \{A_0, A_1, \dots, A_{n-1}\}$ be an arc spline consisting of n segments and (r_i, l_i) is radius and length of the segment A_i of the arc spline, and then the arc spline is a k-arc spline if and only if $(r_i, l_i) \in \{(r^0, l^0), (r^1, l^1), \dots, (r^{k-1}, l^{k-1})\}$ ($i = 0, \dots, n-1$) and $2 \leq k < n$.

Note that superscript of radius r and length l denote indices of clusters and their subscript denote indices of segments of an arc spline in the following discussion. As is clear from the above definition, a k-arc spline includes only k types of arcs that are less than the total number of segments and each segment A_i belongs to one of the clusters C^j . The map $\lambda: \{0, 1, \dots, n-1\} \rightarrow \{0, 1, \dots, k-1\}$ represents the relation between segments and clusters. k-arc splines should be represented by the representative values of radii and arc length, i.e., r^j, l^j ($j = 0, 1, \dots, k-1$). Therefore, we extend the representation of general arc splines (1) to (3) by introducing the discrete variable $\delta_{i,j}$ defined as follows.

$$\delta_{i,j} = \begin{cases} 1 & (\lambda(i) = j) \\ 0 & (\lambda(i) \neq j) \end{cases} \quad (i = 0, 1, \dots, n-1, j = 0, 1, \dots, k-1) \quad (4)$$

Followed the above definition, (1) to (3) are extended to k-arc splines as follows.

$$\mathbf{P}_{i+1} = \mathbf{P}_i + \sum_j \delta_{i,j} r^j (\mathbf{e}_{i+1}^1 - \mathbf{e}_i^1) \times \mathbf{e}_i^3 \quad (i = 0, 1, \dots, n-1) \quad (5)$$

$$\mathbf{P}_i^m = \mathbf{P}_i + \sum_j \delta_{i,j} r^j (\mathbf{e}_i^3 \times \mathbf{e}_i^1) - \sum_j \delta_{i,j} r^j \frac{\mathbf{e}_{i+1}^1 - \mathbf{e}_i^1}{\sqrt{2(1 - \langle \mathbf{e}_{i+1}^1, \mathbf{e}_i^1 \rangle)}} \quad (i = 0, 1, \dots, n-1) \quad (6)$$

$$\mathbf{e}_{i+1}^1 = R\left(\mathbf{e}_{i,z}^3 \frac{\sum_j \delta_{i,j} l^j}{\sum_j \delta_{i,j} r^j}\right) \mathbf{e}_i^1 \quad (i = 0, 1, \dots, n-1) \quad (7)$$

Similar to general arc splines in the previous section, if we are given an initial condition, radii of all clusters, lengths of them, and the relation between all segments and the clusters, we get all nodes and middle points of the k-arc spline. Then, the derivative vectors of (5) to (7) with respect to r^j and l^j are also derived in recurrence formulas.

3. Approximation algorithm

With the representation of k-arc splines, we can construct a k-arc spline approximation algorithm. Given a target curve, a module length of arc segments, and the number of types of arc segments, our algorithm finds a k-arc spline with as high approximation accuracy as possible by solving an optimization problem. Fig. 4 illustrates the overall workflow of the algorithm.

3.1. Optimization formulation

3.1.1. Approximation energy

We evaluate the approximation accuracy of k-arc splines by the sum of norm squares of residual vectors at sampling points. We use all nodes of a k-arc spline and the midpoints of all segments as the sampling points because an arc curve is determined by three passing points. Therefore, the approximation energy is formulated as follows,

$$E = \sum_{i=0}^n d(\tilde{\mathbf{P}}_i, \mathbf{P}_i)^2 + \sum_{i=0}^{n-1} d(\tilde{\mathbf{P}}_i^m, \mathbf{P}_i^m)^2 \quad (8)$$

Where $\mathbf{P}_i$ is a node of the k-arc spline, $\mathbf{P}_i^m$ is a midpoint of each segment, and $\tilde{\mathbf{P}}_i$ and $\tilde{\mathbf{P}}_i^m$ defined in detail in 3.2 are target points of $\mathbf{P}_i$ and $\mathbf{P}_i^m$. Note that $\tilde{\cdot}$ denotes values of target curves in the following discussion.

3.1.2. Interpolation constraints

In actual problems, interpolations at some user-specified points may be required. Here, we show the interpolation constraints at both endpoints. For the G^0 interpolation requirement, the constraints are as follows,

$$\begin{cases} \tilde{\mathbf{P}}_0 - \mathbf{P}_0 = \mathbf{0} \\ \tilde{\mathbf{P}}_n - \mathbf{P}_n = \mathbf{0} \end{cases} \quad (9)$$

We can leave out the constraint at the start point by dealing with $\mathbf{P}_0$ as a constant because $\mathbf{P}_0$ is an independent parameter of k-arc splines in our representation. For the G^1 interpolation requirement, the constraints are as follows,

$$\begin{cases} \tilde{\mathbf{P}}_n - \mathbf{P}_n = \mathbf{0} \\ \frac{1}{\varepsilon_{feas}} (\langle \tilde{\mathbf{e}}_n^1, \mathbf{e}_n^1 \rangle - 1) = 0 \end{cases} \quad (10)$$

For the same reason as above, dealing with $\mathbf{P}_0$ and $\mathbf{e}_0^1$ as constants, we leave out the constraints at the start point. Note that we scale the second constraint by a feasibility tolerance ε_{feas} used in an optimization algorithm for stability and efficiency of optimization computation.

3.1.3. Closed constraints

For closed curves, constraints for continuity are necessary to ensure that arc splines are G^1 continuous closed curves. For closed curves, the constraints are as follows.

$$\begin{cases} \mathbf{P}_n - \mathbf{P}_0 = \mathbf{0} \\ \frac{1}{\varepsilon_{feas}} (\langle \mathbf{e}_n^1, \mathbf{e}_0^1 \rangle - 1) = 0 \end{cases} \quad (11)$$

In this case, no variables are constant. As same with the G^1 interpolation constraints, we scale the second constraint for tangent continuity by ε_{feas} .

3.1.4. Side constraints

In some cases, we would like to control the shape of arc segments directly. In these cases, the following side constraints are introduced.

$$r_{min}^j \leq r^j \leq r_{max}^j, \quad l_{min}^j \leq l^j \leq l_{max}^j \quad (12)$$

Where r_{min}^j , r_{max}^j , l_{min}^j , and l_{max}^j are the user-specified values for the j-th arc. r_{min}^j and l_{min}^j must be positive because we assume only positive radii and lengths. Even if specific ranges are not set, we set the upper and lower bounds to $r^0 \sim r^{k-1}$

and $l^0 \sim l^{k-1}$ to prevent errors in optimization algorithms. In this case, we set a small value, e.g., $\epsilon \approx 1.0e - 6$, to $l_{\min}^j$ and $r_{\min}^j$ and a large value, e.g., $\gamma \approx 1.0e + 12$, to $l_{\max}^j$ and $r_{\max}^j$. Practically, however, solutions rarely reach these default bounds because our initialization algorithm in 3.2 can find a well estimated initial solution.

3.1.5. Optimization

The approximation energy function and some of the constraints form a nonlinear constrained optimization problem as follows.

$$(P) \quad \begin{array}{l} \text{find } \mathbf{P}_0, \mathbf{e}_0^1, r^0, \dots, r^{k-1}, l^0, \dots, l^{k-1}, \delta_{0,0}, \dots, \delta_{n-1,k-1}, \mathbf{e}_0^3, \dots, \mathbf{e}_{n-1}^3, n \\ \text{Min } E \\ \text{s.t. } \Delta r_{\min}^j \leq \Delta r^j \leq \Delta r_{\max}^j \quad (j = 0, 1, \dots, k-1) \\ \Delta l_{\min}^j \leq \Delta l^j \leq \Delta l_{\max}^j \quad (j = 0, 1, \dots, k-1) \\ h_\xi = 0 \quad (\xi = 0, 1, \dots) \quad \text{if necessary. See 3.1.2 or 3.1.3} \end{array} \quad (13)$$

Where $h_\xi = 0$ is a constraint defined in 3.1.2 or 3.1.3. In our representation of k-arc splines, $\mathbf{P}_0, \mathbf{e}_0^1, r^j, l^j$ ($j = 0, 1, \dots, k-1$) are continuous parameters, and $\delta_{i,j}$ ($i = 0, 1, \dots, n-1, j = 0, 1, \dots, k-1$), $\mathbf{e}_i^3$ ($i = 0, 1, \dots, n-1$) and n are discrete parameters. Therefore, problem (13) is a nonlinear mixed-integer problem and cannot be solved efficiently. To solve efficiently, we use a special approach to estimate all discrete parameters initially and set the estimated values as constants in (13). If all discrete parameters are constants, the problem reduces to a continuous optimization problem and can be solved efficiently by conventional optimization algorithms. We describe the estimation method in 3.2.

We solve the continuous optimization problem by the Quasi-Newton method or the SQP method (Nocedal and Wright, 2006). For an unconstrained case, i.e., with only side constraints, we use the Quasi-Newton method. If a solution reaches the bound of a side constraint, an update vector is projected to the feasible region. For a constrained case, we use the SQP method with a line search algorithm. In our implementation, we solve two sub quadratic problems, which are different in their quadratic matrix, in each iteration step to get the better update direction (please see Yamashita et al., 2005 for more details). Both methods require the initial solution, and the quality of the initial solution has a great effect on the computation time. We describe the method to estimate the good initial solution in 3.2.

3.1.6. Perturbation variables

In both optimization algorithms, we solve some linear systems in each iteration step to get updated values of optimization variables. High irregularity in the scale of variables causes instability of linear systems. In actual problems of architectural design, the following irregularity arises, especially when discretizing curves with low curvature values by short segments.

$$l^0 \approx \dots \approx l^{k-1} \ll r^0 < \dots < r^{k-1} \quad (14)$$

Therefore, we introduce the following perturbation variables to normalize variables,

$$\begin{aligned} \mathbf{P}_0 &= \mathbf{P}_{0,\text{initial}} + \Delta \mathbf{P}_0 \\ \mathbf{e}_0^1 &= R(\Delta \theta_0) \mathbf{e}_{0,\text{initial}}^1 \\ r^j &= \Delta r^j \times r_{\text{initial}}^j \quad (j = 0, 1, \dots, k-1) \\ l^j &= \Delta l^j \times l_{\text{initial}}^j \quad (j = 0, 1, \dots, k-1) \end{aligned} \quad (15)$$

Where $\mathbf{P}_{0,\text{initial}}, \mathbf{e}_{0,\text{initial}}^1, r_{\text{initial}}^j$ and l_{initial}^j are each initial value. With the introduction of perturbation variables, it is necessary to slightly modify the representations (5) to (7) of k-arc splines. Additionally, side constraints (12) must be represented by Δr^j and Δl^j . Consequently, we solve the following problem.

$$(P) \quad \begin{array}{l} \text{find } \Delta \mathbf{P}_0, \Delta \theta_0, \Delta r^0, \dots, \Delta r^{k-1}, \Delta l^0, \dots, \Delta l^{k-1} \\ \text{Min } E \\ \text{s.t. } \Delta r_{\min}^j \leq \Delta r^j \leq \Delta r_{\max}^j \quad (j = 0, 1, \dots, k-1) \\ \Delta l_{\min}^j \leq \Delta l^j \leq \Delta l_{\max}^j \quad (j = 0, 1, \dots, k-1) \\ h_\xi = 0 \quad (\xi = 0, 1, \dots) \quad \text{if necessary.} \end{array} \quad (16)$$

3.2. Initialization

In the previous subsection, the k-arc spline approximation is formulated as a continuous optimization problem. We need appropriately estimated discrete parameters, i.e., $\delta_{i,j}, \mathbf{e}_i^3$ and n , to construct the problem. Additionally, to apply the

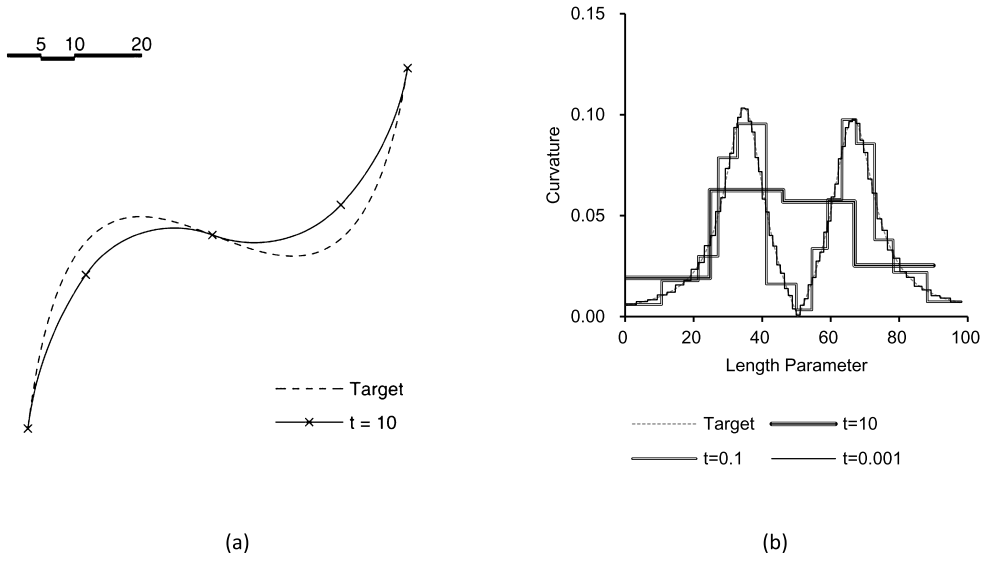


Fig. 3. The relation between approximation accuracy of curve and curvature in the general arc spline approximation method implemented in Rhinoceros®. (a) shows curve appearance of the target curve and the arc spline with deviation tolerance = 10. (b) shows curvature plot of the target curve and some arc splines of various tolerance values. The smaller the tolerance is, the closer the curvature becomes to the curvature of the target curve.

optimization algorithms, we need the initial values for continuous parameters, i.e., $\mathbf{P}_{0,initial}$, $\mathbf{e}_{0,initial}^1$, $r_{initial}^j$, and $l_{initial}^j$. For this initialization, we use curvature values of target curves.

3.2.1. Curvature clustering algorithm

From the fundamental theorem of plane curves, a plane curve can be uniquely determined if a curvature $\kappa(s)$ is determined, except for the rigid transformation. From this theorem, if an arc spline that accurately approximates a target curve is given, it is assumed that the curvature of the arc spline is close to the curvature of the target curve. The same viewpoint is described in Piegls et al. 2002, and we can confirm this property by a general arc spline approximation method shown in Fig. 3. Hence, based on this assumption, instead of the curve approximation problem, we solve the curvature approximation problem to estimate appropriate values for discrete parameters and the initial values for continuous parameters.

A curvature function of k-arc splines is a step function like general arc splines, but it has the following additional properties,

- The step function has only k types of values
- The interval length of each step of the same curvature value is an integral multiple of a specific reference length, i.e., module.

We thus use the following Curvature Clustering Algorithm (CCA) to approximate the curvature function of a target curve with the above function.

1. Split the target curve at inflection points into global segments.
2. Divide each global segment equally into local segments by the module length M .
3. Get constant curvature values in local segments.
4. Cluster the local curvature values by k-means⁺⁺.

In the first step, we split the target curve at inflection points because binormal vectors reverse at those points, and direction changes of binormal vectors affect the global shape of curves. In the second step, each global segment is divided equally into local segments. The divide count of each global segment is calculated as $\text{Round}(L/M)$, where L is the length of each global segment and M is a user-specified module length. We introduce the module length M to get uniform local segments because the length $l^0, l^1, \dots, l^{k-1}$ should be approximately equal considering the application in architectural engineering. More technically speaking, when paneling curved surfaces, it is generally preferable to divide them into panels of uniform size from viewpoints of both design and engineering. We assume that the local segments obtained in this step correspond to ones of a k-arc spline. Based on this assumption, the number of segments of the k-arc spline n and the binormal vectors $\mathbf{e}_i^3$ are determined. Additionally, $\tilde{\mathbf{P}}_i$ and $\tilde{\mathbf{P}}_i^m$ in (8) are defined as the endpoints and midpoints of the local segments. These target points are fixed during the optimization procedure. In the third step, we average curvature values in each local segment to get constant curvature values. Using the average curvature values $\bar{\kappa}_0, \bar{\kappa}_1, \dots, \bar{\kappa}_{n-1}$ and the module length M , we

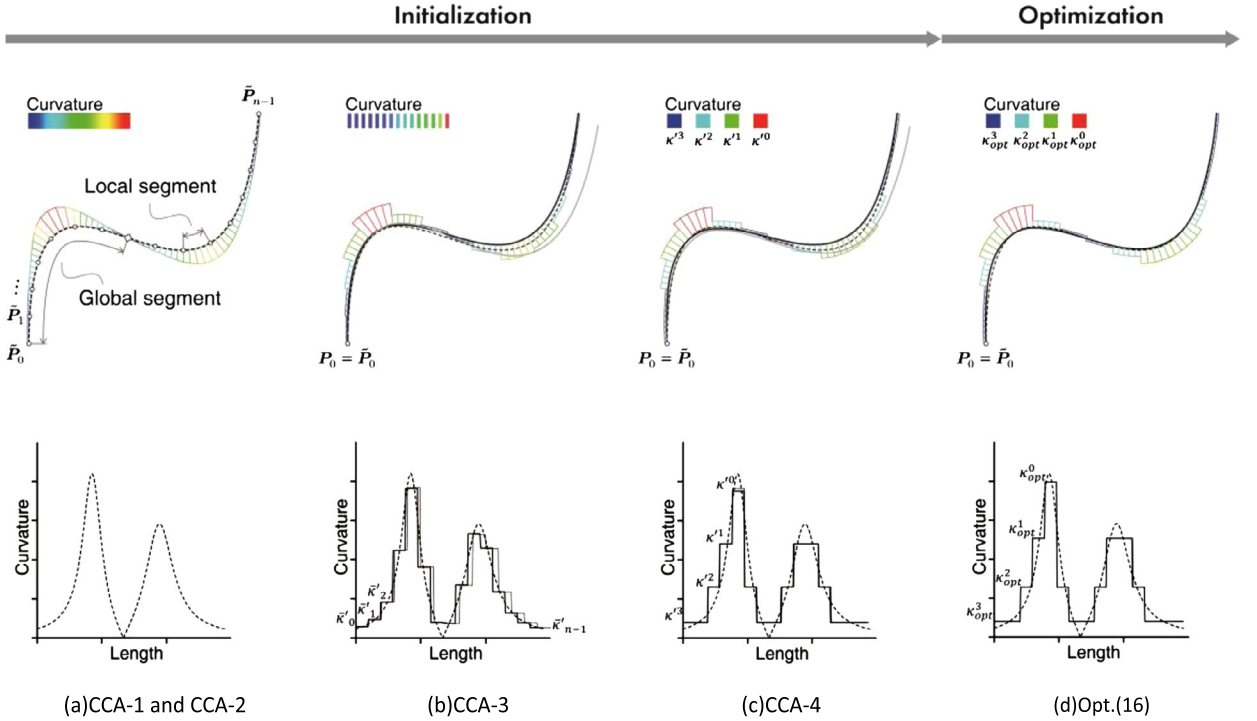


Fig. 4. The overall workflow of the k-arc spline approximation algorithm. (a) shows the segmentation process. (b) and (c) show the process of constructing a new step function of curvature. In the figures of both steps, the black dashed curve is the target, the gray solid line is a curve without the fast improvement method and the black solid one is a curve with the fast improvement method. (d) shows the final result.

get a step function of curvature. This function approximates the target curvature function well, and the curve defined by this function, i.e., a general arc spline, approximates the target curve with reasonable accuracy. If we solve a curve approximation problem for the general arc spline, we can improve the step function considering curve approximation accuracy. However, solving the same optimization problem as (16) with general arc splines is computationally expensive because the number of variables is much higher than the problem with k-arc splines. Therefore, we solve an interpolation problem, i.e., a simple nonlinear equation, to improve the step function. This fast improvement method is described in 3.2.2. Applying this method, we get improved curvature values $\bar{\kappa}'_0, \bar{\kappa}'_1, \dots, \bar{\kappa}'_{n-1}$ and segment lengths $l'_0 = l'_1 = \dots = l'_{n-1} = M'$. In the last step, $\bar{\kappa}'_0, \bar{\kappa}'_1, \dots, \bar{\kappa}'_{n-1}$ are clustered into k groups by the k-means⁺⁺ method in Arthur and Vassilvitskii (2007). The k-means⁺⁺ method partitions the values into k sets to minimize within-cluster sum of squares. Therefore, if we replace all values of the step function in the previous step with the centroidal value of the corresponding cluster, the new step function has only k types of values $\kappa^0, \kappa^1, \dots, \kappa^{k-1}$ and approximates the original curvature function well. As with the previous step, we apply the fast improvement method and get improved curvature values $\kappa'^0, \kappa'^1, \dots, \kappa'^{k-1}$ and representative lengths $l'^0, l'^1, \dots, l'^{k-1}$. Therefore, we can determine $r_{initial}^j = 1/\kappa'^j$ and $l_{initial}^j = l'^j$. Additionally, we can determine $\delta_{i,j}$ from the partitioning result by k-means⁺⁺.

Consequently, given the target curve and the module length M , the following values are determined through the overall algorithm,

$$n, \mathbf{e}_i^3, \delta_{i,j}, r_{initial}^j, l_{initial}^j \quad (i = 0, \dots, n-1, j = 0, \dots, k-1)$$

3.2.2. Fast improvement

In the curvature clustering algorithm, we construct a step function from a curvature function of a target curve (step 3) or another step function of curvature (step 4). When constructing a new step function from another curvature function, we consider only curvature approximation accuracy. Therefore, an arc spline with the new curvature function is not the best-approximated curve. If we solve an optimization problem like (16) every time constructing a new curvature function, we always get the optimal arc spline. This greedy strategy, however, is computationally too expensive. Hence, we adopt a less accurate but more efficient approach.

Given a step function of curvature, and a boundary condition, i.e., the initial point $\mathbf{P}_0$ and the initial tangent vector $\mathbf{e}_0^1$, we can construct an arc spline. If the step function approximates a curvature function of a target curve well and we use the same boundary condition with the target curve, the arc spline approximates the target curve with reasonable accuracy. Approximation accuracy, however, deteriorates as it approaches the endpoint because the arc segments are generated

sequentially from the initial point. Therefore, we solve an interpolation problem only at the endpoint to improve approximation accuracy efficiently. Since this problem is just a non-linear equation, it can be solved much more efficiently than an optimization problem.

In case that the final interpolation requirement is nothing or G^0 at both endpoints for open curves, the following equation is solved with the initial point fixed.

$$\langle \tilde{\mathbf{P}}_n - \mathbf{P}_n, \tilde{\mathbf{P}}_n - \mathbf{P}_n \rangle = 0 \quad (17)$$

In case that the final requirement is G^1 interpolation at both endpoints, the following equation is solved with the initial point and tangent fixed.

$$\begin{cases} \langle \tilde{\mathbf{P}}_n - \mathbf{P}_n, \tilde{\mathbf{P}}_n - \mathbf{P}_n \rangle = 0 \\ \langle \tilde{\mathbf{e}}_n^1, \mathbf{e}_n^1 \rangle - 1 = 0 \end{cases} \quad (18)$$

Finally, for closed curves, the following equation is solved without any constraints.

$$\begin{cases} \langle \mathbf{P}_n - \mathbf{P}_0, \mathbf{P}_n - \mathbf{P}_0 \rangle = 0 \\ \langle \mathbf{e}_n^1, \mathbf{e}_0^1 \rangle - 1 = 0 \end{cases} \quad (19)$$

Note that the squared residual norm function and the non-scaled tangent vector condition are adopted in contrast to the optimization constraints in 3.1.2 and 3.1.3 because these can be solved correctly when solving as simple nonlinear equations. If we solve the above equations in CCA-Step3, we use representations of general arc splines (1)–(3) to control all curvature values of local segments. As for segment lengths, we use a common variable for all segments for uniformity. In CCA-Step4, we use representations of k-arc splines (4)–(6).

Each interpolation condition is a nonlinear equation, that is, $F(\mathbf{x}) = \mathbf{0}$, but the number of equations is smaller than the number of unknown variables, and we cannot apply the usual Newton-Raphson method directly. If we use the usual one, we must set appropriate additional conditions. However, it is difficult to define proper conditions that reliably improve the initial solution. Therefore, we apply the generalized Newton-Raphson method that enables us to solve the equations without additional conditions (Kawaguchi, 2001). In this method, the following linear equation is solved to obtain an update vector at each step as with the usual one.

$$J(\mathbf{x}_i) \Delta \mathbf{x}_i = -\mathbf{r}_i \quad (20)$$

Where J is a jacobian matrix and $\mathbf{r}_i$ is the residual vector at the step. The generalized Newton-Rapson method solves the above equation directly by the generalized inverse matrix J^+ . Since the update vector $\Delta \mathbf{x}_i = -J^+ \mathbf{r}_i$ is the minimum norm solution, the norm of $\Delta \mathbf{x}_i$ is smaller than a solution obtained by introducing additional constraints. There is no guarantee that a converged solution $\mathbf{x}_{sol}$ after the iterative calculations is the minimum norm solution, but the norm of $\mathbf{x}_{sol} - \mathbf{x}_0$ tends to be smaller than that of the usual method. Because of this minimum norm property, we get an interpolated arc spline that maintains its initial step function of curvature as much as possible. Therefore, we get a good arc spline in aspects of both curve approximation and curvature approximation.

As with the optimization problem, we use perturbation variables as unknowns to prevent highly biased solutions. In CCA-Step3, we introduce perturbation variables for $\mathbf{P}_0$, $\mathbf{e}_0^1$, r_i ($i = 0, 1, \dots, n-1$), l to control radii of all segments and a common length (see (16) for each definition). In CCA-Step4, we use the same perturbation variables as (16). If using the original variables directly as unknowns in irregular cases with (14), J and J^+ become highly biased matrices. As a result, update vectors in all iteration steps are biased, and the converged solution is also biased.

In addition to improvement of approximation accuracy, we check feasibility of the optimization problem (16) in this method. If the degree of freedom of a k-arc spline is low, i.e., k is small, the equation (17)–(19) may not have any solution. In this case, the optimization problem (16) with the interpolation constraints or closed constraints also has no feasible solution. Therefore, we can determine feasibility of (16) by monitoring the convergence of the generalized Newton-Raphson method in CCA-Step4. If not converge, the method returns the input value directly, i.e., the curvature function is not modified, and excludes the constraints from (16) to prevent the non-convergence of SQP. Eventually, we solve the unconstrained problem and get a compromised solution in the optimization step.

4. Results and discussions

4.1. Results

In this section, we show some examples include both open and closed curves. All numerical tests were performed on a laptop PC with an Inter® Core™ i9-9980HK CPU. Table 1 Shows approximation parameters for all cases. For open curves, we adopt the G^1 interpolation constraint (10). The results are shown in Fig. 5 and Fig. 6. In both cases, the optimization problem converges successfully, and the resulting curve well-approximates the target curve under our parameter settings. The curvature plot also approximates the curvature plot of the target well, indicating high approximation quality. Approximation accuracy is lower than general arc splines due to strict constraints on curvature and length but sufficient for architectural

Table 1

k-arc spline approximation parameters. In actual applications, k and M are set by users considering the requirements of each project. ϵ_{OPT} and ϵ_{FEAS} are tolerance values of the optimization problem and affect approximation accuracy. ϵ_{FEAS} is related to interpolation accuracy and we set the value considering a general tolerance value of CAD software.

	Open-A	Open-B	Closed-A	Closed-B
k	5	6	6	9
M	4.0	2.0	2.0	1.0
ϵ_{OPT}	0.01	0.01	0.01	0.01
ϵ_{FEAS}	1.0×10^{-4}	1.0×10^{-4}	1.0×10^{-4}	1.0×10^{-4}

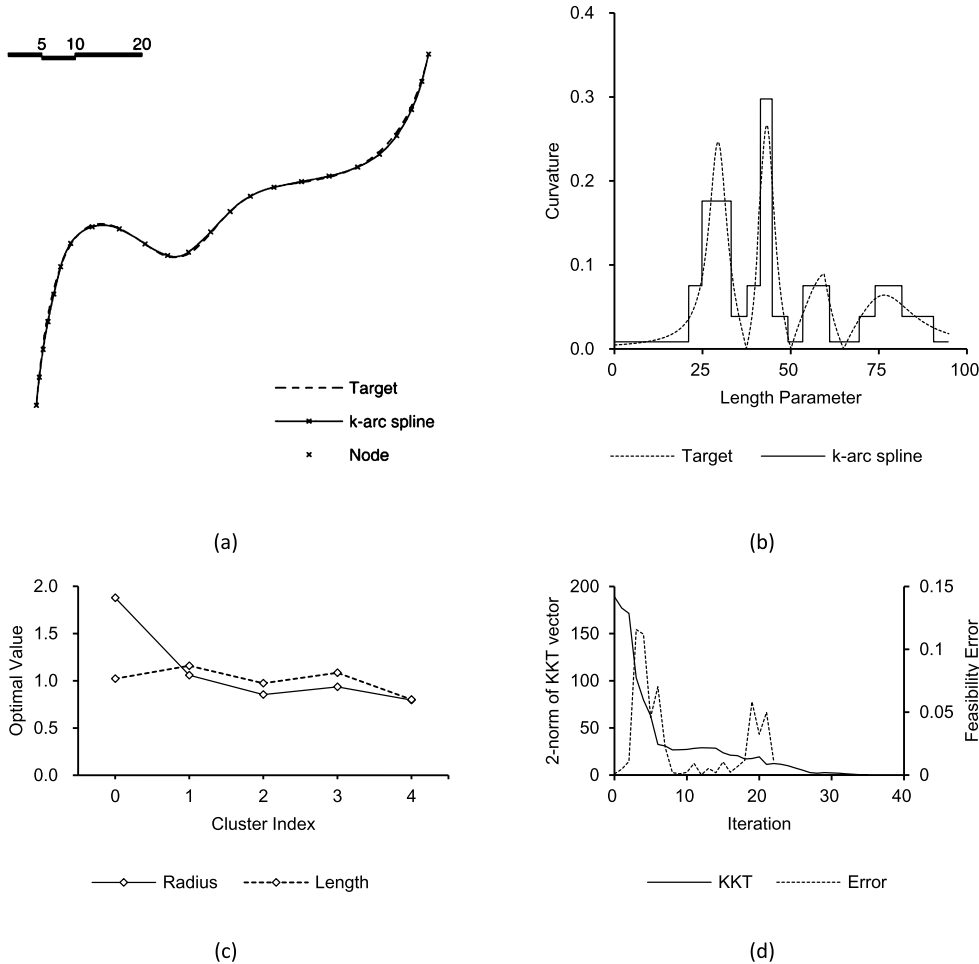


Fig. 5. The results of Open-A. (a) Curve appearance. (b) Curvature plot. (c) Optimal solutions. (d) Optimality and feasibility history during optimization process.

engineering applications. For closed curves, we adopt the closed constraint. The results are shown in Fig. 7 and Fig. 8 and indicate that our algorithm can be applied properly to closed curves as well as open curves. Furthermore, Fig. 8 shows that our algorithm is also applicable to curves with a large range of curvature, such as Closed-B.

4.2. Discussion

In all cases, an approximate curve was obtained in seconds to tens of seconds, and most of the calculation time was spent on the optimization process. Table 2 shows the detailed results of the optimization process. For fast but accurate computation, setting the appropriate optimality tolerance ϵ_{OPT} is required. In the optimization algorithm, we scale the ϵ_{OPT} based on Yamashita et al., 2005, but the standard value greatly affects computation time. Therefore, we tested some tolerance values on some curves, and Fig. 9 shows one of the results. If the tolerance is too small, the results will be almost the same, but the calculation time will increase drastically. Consequently, 0.01 to 0.1 is suitable for most curves (see also Table 3).

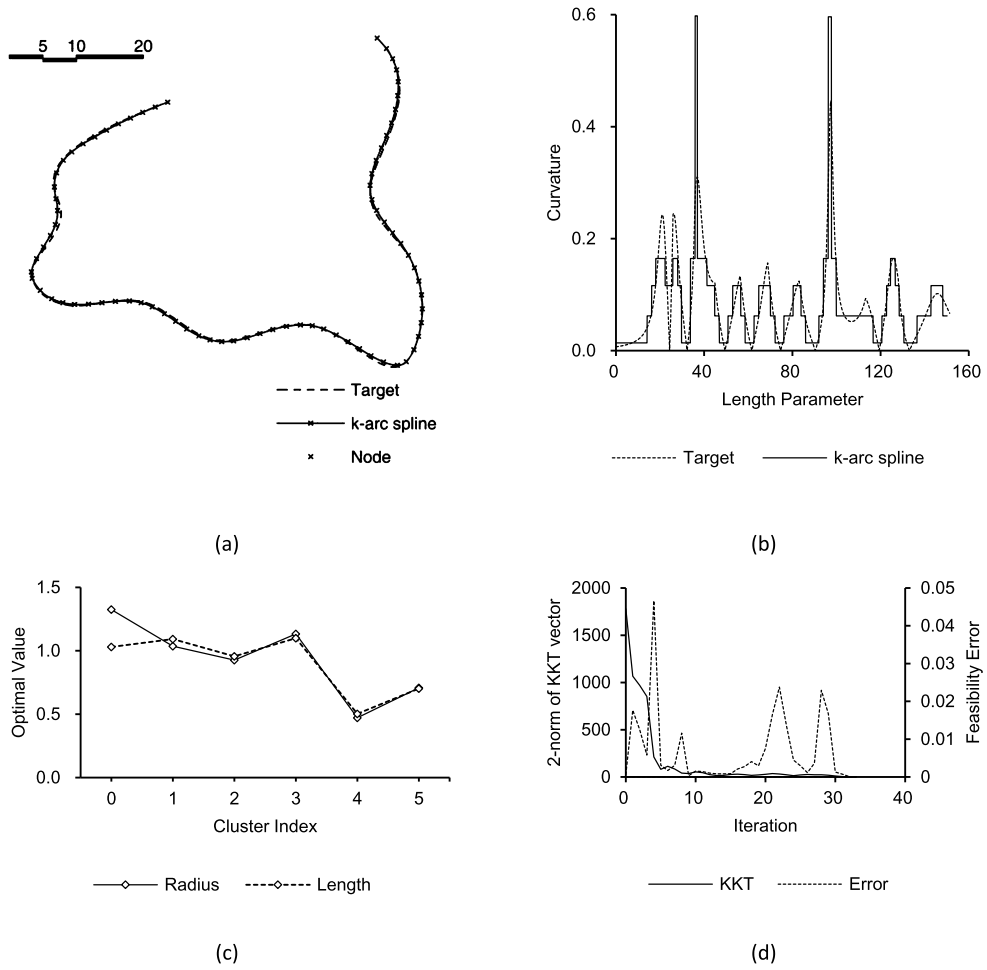


Fig. 6. The results of Open-B. (a) Curve appearance. (b) Curvature plot. (c) Optimal solutions. (d) Optimality and feasibility history during optimization process.

Table 2

Detailed results of the optimization process. Calculation time per step depends on the number of segments and the number of clusters k because of the high cost of evaluation of derivative functions.

	Open-A	Open-B	Closed-A	Closed-B
Number of segments n	23	76	79	167
Final KKT vector norm	0.415	2.21	0.547	3.03
Final feasibility error	0.000046	0.000044	0.000002	0.000046
Iteration	35	33	75	74
Total time [s]	0.86	3.8	10.7	40.4

Table 3

Detailed results of the optimization process of the tolerance test on Open-B. Max deviation is evaluated by hausdorff distance between the target curve and the k-arc spline.

	Initial	$\epsilon_{OPT} = 1.0$	$\epsilon_{OPT} = 10^{-2}$	$\epsilon_{OPT} = 10^{-4}$	$\epsilon_{OPT} = 10^{-6}$
KKT vector norm	1780.1	2.21	2.21	1.40e-1	3.76e-4
Feasibility error	6.84e-4	4.4e-5	4.4e-5	2.6e-8	7.3e-9
Max deviation	2.77	0.547	0.547	0.543	0.542
Iteration	–	33	33	53	68
Total time[s]	–	3.8	3.8	6.5	8.4

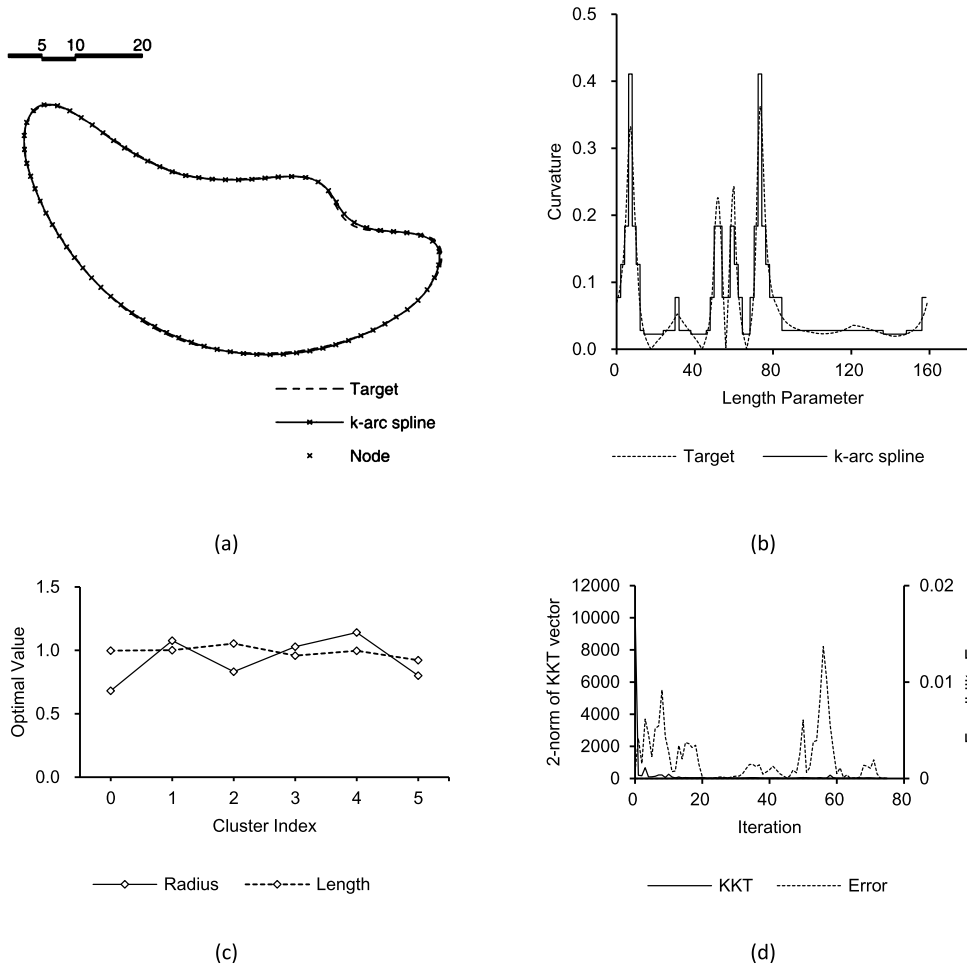


Fig. 7. The results of Closed-A. (a) Curve appearance. (b) Curvature plot. (c) Optimal solutions. (d) Optimality and feasibility history during optimization process.

4.3. Extension

For applications in architectural engineering, in the case of a building composed of multiple curved walls shown in Fig. 10, all curves are required to be composed of a limited number of types of elements. The presented algorithm in this paper can also solve such problems with a simple extension, i.e., clustering the curvatures of all curves together and determining the assignment of k types of arcs to all local segments. Fig. 10 shows the results of this extension.

4.4. Limitations

The presented algorithm determines the assignment pattern of k types of arcs only by curvature values of target curves and dealing with them as constants in the optimization process. Regarding the assignment pattern as optimization variables may give a more well-approximated k -arc spline. However, because this increases calculation cost drastically, an efficient algorithm to solve combinations is required. Furthermore, users are required to set parameters k and M by trial and error while considering the requirements of cost and design quality. A method that can find the optimal k and M within a given tolerance may be necessary.

5. Conclusion

A new arc spline approximation method for application in architectural engineering was proposed. The algorithm uses special arc splines, named k -arc splines, consisting of a limited number of segment types, which is less than the total number of segments. We introduced a representation of k -arc splines, and we formulated an approximation problem by k -arc splines as a continuous optimization problem that conventional algorithms can solve efficiently. Initialization of the problem is the key to obtain a well-approximated k -arc spline, and we showed that clustering of curvature values of a target

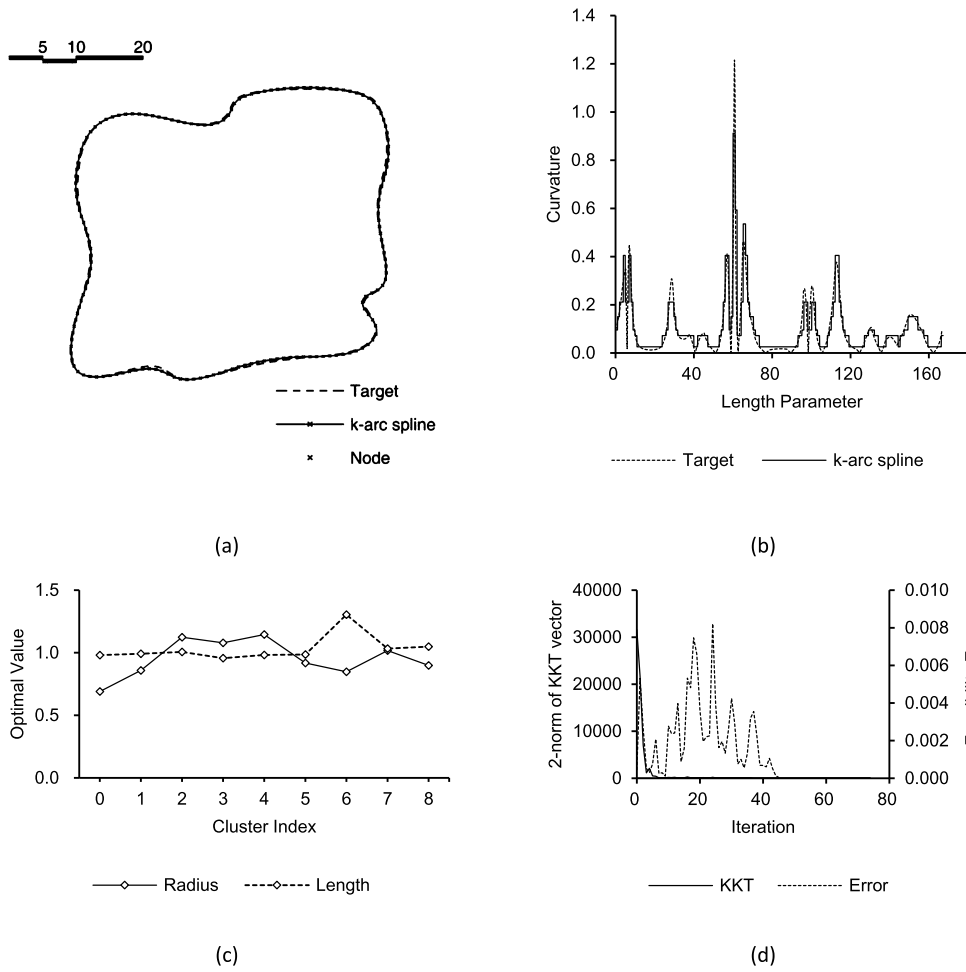


Fig. 8. The results of Closed-B. (a) Curve appearance. (b) Curvature plot. (c) Optimal solutions. (d) Optimality and feasibility history during optimization process.

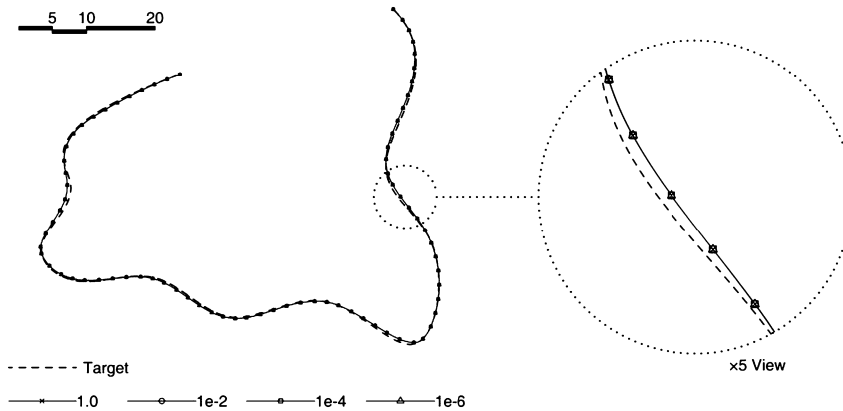


Fig. 9. The results of a tolerance test on Open-B. Curve appearances are almost the same for all tolerances.

curve by the k-means⁺⁺ method performs the proper initialization. We have tested some examples and showed the validity of our algorithm.

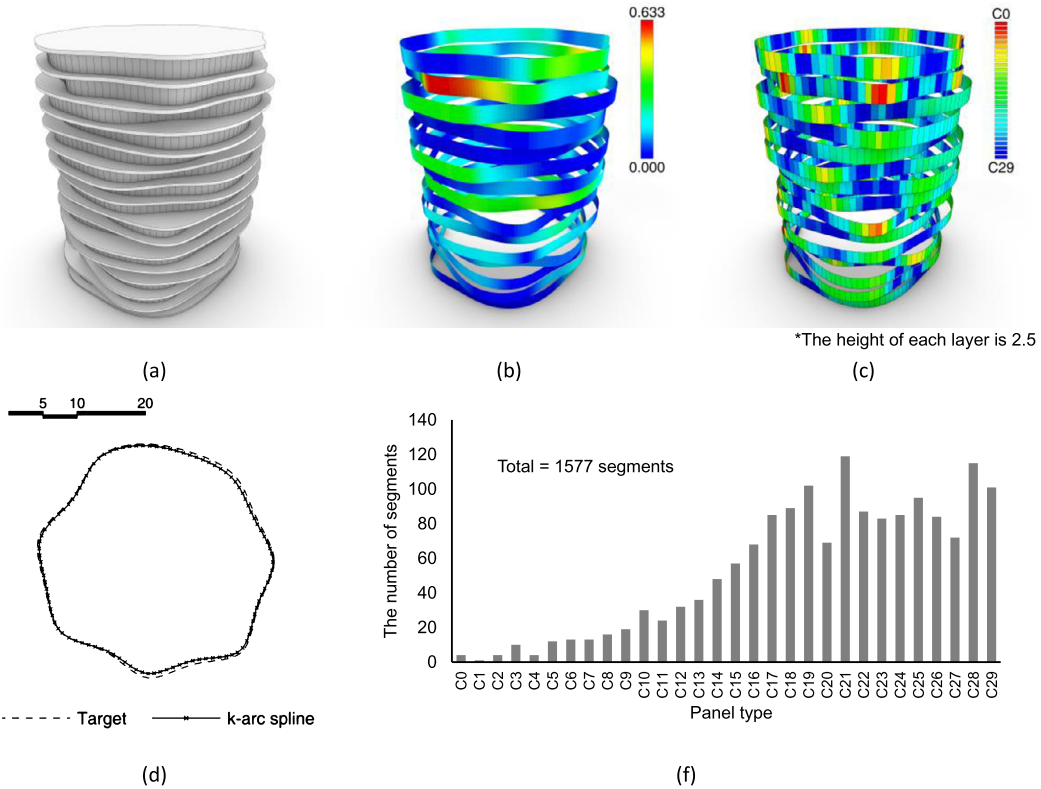


Fig. 10. The results of 15 closed curves created by randomly arranging 15 control points. The number of clusters k is set to 30. (a) A building image with 15 curved walls. (b) Deviation between the target curves and the k -arc splines. (c) Panel assignments. (d) Curve appearance of the curve with the worst approximation accuracy. (e) The number of segments in each cluster. (f) The number of segments in each cluster.

CRediT authorship contribution statement

Keisuke Mizutani: Conceptualization, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Ken'ichi Kawaguchi:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Ahn, Young Joon, Kim, Hong Oh, Lee, Kyoung-Yong, 1998. G^1 arc spline approximation of quadratic Bezier curves. *Comput. Aided Des.* 30 (8), 615–620.
- Arthur, D., Vassilvitskii, S., 2007. k-means++: the advantages of careful seeding. In: *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1027–1035.
- Bolton, K.M., 1975. Biarc curves. *Comput. Aided Des.* 7 (2), 89–92.
- Chen, Xiao-Diao, Yong, Jun-Hai, Zheng, Quo-Qin, Sun, Jia-Guang, 2004. Automatic G^1 arc spline interpolation for closed point set. *Comput. Aided Des.* 36, 1205–1218.
- Eigensatz, M., Kilian, M., Schiftner, A., Mitra, N.J., Pottmann, H., Pauly, M., 2010. Paneling architectural freedom surfaces. *ACM Trans. Graph.*
- Hoschek, J., 1992. Circular splines. *Comput. Aided Des.* 24 (11), 611–618.
- Kawaguchi, K., 2001. *Generalized Inverse and Its Applications to Structural Engineering*. CORONA PUBLISHING CO., LTD.
- Maier, G., 2014. Optimal arc spline approximation. *Comput. Aided Geom. Des.* 31, 211–216.
- Meek, D.S., Walton, D.J., 1992. Approximation of discrete data by G^1 arc splines. *Comput. Aided Des.* 24 (6), 301–306.
- Meek, D.S., Walton, D.J., 1993. Approximating quadratic NURBS curves by arc splines. *Comput. Aided Des.* 25 (6), 371–376.
- Meek, D.S., Walton, D.J., 1995. Approximating smooth planar curves by arc splines. *J. Comput. Appl. Math.* 59, 221–231.
- Nocedal, J., Wright, Stephen J., 2006. *Numerical Optimization*. Springer, New York.
- Ong, C.J., Wong, Y.S., Loh, H.T., Hong, X.G., 1996. An optimization approach for biarc curve-fitting of B-spline curves. *Comput. Aided Des.* 28 (12), 951–959.
- Parkinson, D.B., Moreton, D.N., 1991. Optimal biarc-curve fitting. *Comput. Aided Des.* 23 (6), 411–419.
- Piegl, Les A., Tiller, Wayne, 2002. Biarc approximation of NURBS curves. *Comput. Aided Des.* 34, 807–814.
- Schonherr, J., 1993. Smooth biarc curves. *Comput. Aided Des.* 25 (6), 365–370.
- Walton, D.J., Meek, D.S., 1994. Approximation of quadratic Bezier curves by arc splines. *J. Comput. Appl. Math.* 54, 107–120.

- Yamashita, H., Hiroshi, Y., Takahito, T., 2005. A globally and superlinearly convergent primal-dual interior point trust region method for large scale constrained optimization. *Math. Program.* 102, 111–151.
- Yeung, M.K., Walton, Desmond J., 1994. Curve fitting with arc splines for NC toolpath generation. *Comput. Aided Des.* 26 (11), 845–849.
- Yong, Jun-Hai, Hu, Shi-Min, Sun, Jia-Guang, 1999. A note on approximation of discrete data by G^1 arc splines. *Comput. Aided Des.* 31, 911–915.